

Gradient-Based Optimization Algorithms

CS115.Q11

N. H. Vinh¹ N. H. Kha¹

¹Department of Computer Science
University of Information and Technology

CS115 Presentation

Table of Contents

- 1 Preliminaries
 - Notation
 - Mathematical Formulas
 - Loss Gradient Function
- 2 First-Order Methods
 - Gradient Descent
 - Momentum-Based Methods
 - Adaptive Learning Rate Methods
- 3 Second-Order Methods
 - Newton's Method
 - Quasi-Newton Methods
- 4 Future Work

Table of Contents

1 Preliminaries

- Notation
- Mathematical Formulas
- Loss Gradient Function

2 First-Order Methods

- Gradient Descent
- Momentum-Based Methods
- Adaptive Learning Rate Methods

3 Second-Order Methods

- Newton's Method
- Quasi-Newton Methods

4 Future Work

Preliminaries - Notation

- $\mathbf{x}_i$ is the i -th input vector from the dataset.
- y_i is the corresponding label for $\mathbf{x}_i$.
- N is the total number of samples.
- $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ is the dataset.
- θ is the parameter of the model.
- θ^* is the optimal solution.
- η is the learning rate
- $g(\theta_t; \xi_t)$ is the stochastic gradient on random mini-batch ξ_t
($\mathbb{E}[g(\theta_t; \xi_t)] = \nabla \mathcal{L}(\theta_t; \mathcal{D})$)

Preliminaries - Mathematical Formulas I

- Gradient Vector:

$$\nabla F(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)$$

- Hessian Matrix:

$$\nabla^2 F(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- Jacobian ($f : \mathbb{R}^n \rightarrow \mathbb{R}^m$):

$$J_f(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Preliminaries - Mathematical Formulas II

- Hadamard Product (element-wise product) of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$:

$$\mathbf{a} \odot \mathbf{b} = \begin{bmatrix} a_1 b_1 \\ a_2 b_2 \\ \vdots \\ a_n b_n \end{bmatrix}$$

- Element-wise Square (Hadamard power 2):

$$\mathbf{a}^2 = \mathbf{a} \odot \mathbf{a} = \begin{bmatrix} a_1^2 \\ a_2^2 \\ \vdots \\ a_n^2 \end{bmatrix}$$

Preliminaries - Mathematical Formulas III

- Element-wise Square Root:

$$\sqrt{\mathbf{a}} = \begin{bmatrix} \sqrt{a_1} \\ \sqrt{a_2} \\ \vdots \\ \sqrt{a_n} \end{bmatrix}$$

Preliminaries - Loss Gradient Function I

The gradient function:

$$\nabla L(\theta) = \frac{2}{m} \mathbf{x}^\top (\mathbf{x}\theta + y)$$

```
def grad_func(X, y, theta):  
    m = len(y)  
    gradients = (2/m) * X.T.dot(X.dot(theta) - y)  
    return gradients
```


Table of Contents

- 1 Preliminaries
 - Notation
 - Mathematical Formulas
 - Loss Gradient Function
- 2 **First-Order Methods**
 - Gradient Descent
 - Momentum-Based Methods
 - Adaptive Learning Rate Methods
- 3 Second-Order Methods
 - Newton's Method
 - Quasi-Newton Methods
- 4 Future Work

First-Order Methods

Gradient Descent

Gradient Descent

Update formula:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t, \mathcal{D}),$$

First-Order Methods

Gradient Descent

Algorithm 1 Gradient Descent

Require: η : Learning rate

Require: $\mathcal{L}(\theta)$: Loss function

Require: θ_0 : Initial parameter

1: $t \leftarrow 0$

2: **while** not converged **do**

3: $g_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_t)$

4: $\theta_{t+1} \leftarrow \theta_t - \eta g_t$

5: $t \leftarrow t + 1$

6: **end while**

7: **return** θ_{t+1}

- ▷ Initialize step
- ▷ Compute gradient
- ▷ Update parameter
- ▷ Update step

First-Order Methods

Gradient Descent

```
def gradient_descent(grad_fn, theta0, eta, T, X, y):  
    for t in range(T):  
        g = grad_func(X, y, theta)  
        theta0[:] = theta0 - eta * g
```

First-Order Methods

Gradient Descent

Problems:

- Use all training data, can be really **inefficient** with huge dataset.
- Cannot escape **local minima, saddle points**.
- The **lack of generalization** ability is due to the fact that large-batch methods tend to converge to sharp minimizers of the training function.

First-Order Methods

Stochastic Gradient Descent

Stochastic Gradient Descent

Update formula:

$$\theta_{t+1} = \theta_t - \eta g(\theta_t, \xi_t),$$

- Uses only a small subset of data (mini-batch or single sample) at each update, leading to **faster computation**.
- Adds **stochastic noise** to the gradient, which helps the optimizer **escape local minima** and explore the loss landscape.

First-Order Methods

Stochastic Gradient Descent

Algorithm 2 Stochastic Gradient Descent (SGD)

Require: η : Learning rate

Require: $\mathcal{L}(\theta)$: Loss function

Require: θ_0 : Initial parameter

1: $t \leftarrow 0$

2: **while** not converged **do**

3: Sample ξ_t

▷ Randomly chosen data point

4: $g_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_t; \xi_t)$

▷ Stochastic gradient

5: $\theta_{t+1} \leftarrow \theta_t - \eta g_t$

▷ Parameter update

6: $t \leftarrow t + 1$

7: **end while**

8: **return** θ_{t+1}

First-Order Methods

Stochastic Gradient Descent

```
def stochastic_gradient_descent(grad_fn, theta0, eta, T, X, y):  
    theta = theta0  
    m = len(y)  
    for t in range(T):  
        i = np.random.randint(0, m)  
        X_i = X[i:i+1]  
        y_i = y[i:i+1]  
        g = m*grad_func(X_i, y_i, theta)  
        theta = theta - eta * g  
    return theta
```


First-Order Methods

Stochastic Gradient Descent

Problems:

- Gradient estimates are noisy $\Rightarrow$ cause oscillations near the optimum.
- High variance in updates $\Rightarrow$ slower and less stable convergence.
- May get trapped in local minima or saddle points.
- Highly sensitive to learning rate and not adaptive to feature scaling.

First-Order Methods - Momentum-Based Methods

Heavy-Ball

Heavy Ball

Update formula:

$$\theta_{t+1} = \theta_t - \eta g(\theta_t; \xi_t) + \beta(\theta_t - \theta_{t-1}) \iff \begin{cases} \gamma_{t+1} = \beta\gamma_t + \eta g(\theta_t; \xi_t) \\ \theta_{t+1} = \theta_t - \gamma_{t+1} \end{cases}$$

where

- $0 \leq \beta < 1$: momentum coefficient (scales the previous velocity),
- Introduce momentum $\Rightarrow$ Faster convergence.
- Introduce momentum $\Rightarrow$ Help escape local minima.

First-Order Methods - Momentum-Based Methods

Heavy-Ball

Problems:

- How does β affect the optimizer?
- How does velocity affect the optimizer?

First-Order Methods - Momentum-Based Methods

Nesterov Accelerated Gradient

NAG - Nesterov Accelerated Gradient

Update formula:

$$\begin{cases} \gamma_{t+1} = \theta_t - \eta g(\theta_t; \xi_t) \\ \theta_{t+1} = \gamma_{t+1} + \beta(\gamma_{t+1} - \gamma_t) \end{cases} \iff \begin{cases} m_{t+1} = \beta m_t - \eta g(\gamma_t + \beta m_t; \xi_t) \\ \gamma_{t+1} = \gamma_t + m_{t+1} \end{cases}$$

where

- $m_{t+1} = \gamma_{t+1} - \gamma_t$, with $m_0 = 0, \gamma_0 = \theta_0$
- $0 \leq \beta < 1$: momentum coefficient (scales the previous velocity),
- **Faster convergence:** anticipates future position, correcting the direction earlier than standard momentum.
- **Reduced oscillations:** smoother trajectory near minima thanks to lookahead gradient.

First-Order Methods - Momentum-Based Methods

Nesterov Accelerated Gradient

Problems:

- Fine-tune hyperparameters.

First-Order Methods - Momentum-Based Methods

Unified Momentum

Unified Momentum

Update formula:

$$\text{UM} : \begin{cases} \gamma_{t+1} = \theta_t - \eta g(\theta_t; \xi_t) \\ \gamma_{t+1}^s = \theta_t - s\eta g(\theta_t; \xi_t) \\ \theta_{t+1} = \gamma_{t+1} + \beta(\gamma_{t+1}^s - \gamma_t^s) \end{cases}$$

where

- $0 \leq \beta < 1$: momentum coefficient,
- $s \geq 0$: parameter controlling the variant (e.g., $s = 0$ gives Heavy Ball, $s = 1$ gives NAG)

First-Order Methods - Momentum-Based Methods

Unified Momentum

Algorithm 3 Unified Momentum (UM)

Require: η : Learning rate

Require: β : Momentum coefficient

Require: s : Scaling factor

Require: $\mathcal{L}(\theta)$: Loss function

Require: θ_0 : Initial parameter

1: $t \leftarrow 0$

2: **while** not converged **do**

3: $\gamma_{t+1} \leftarrow \theta_t - \eta g(\theta_t; \xi_t)$

4: $\gamma_{t+1}^s \leftarrow \theta_t - s\eta g(\theta_t; \xi_t)$

5: $\theta_{t+1} \leftarrow \gamma_{t+1} + \beta(\gamma_{t+1}^s - \gamma_t^s)$

6: $t \leftarrow t + 1$

7: **end while**

8: **return** Final parameter θ_T

First-Order Methods - Momentum-Based

Unified Momentum

```
def unified_momentum(grad_func, theta0, eta, beta, s, T, X, y):
    gamma = theta0.copy()
    gamma_s = theta0.copy()
    m = len(y)
    for t in range(T):
        i = np.random.randint(0, m)
        X_i = X[i:i+1]
        y_i = y[i:i+1]
        g = grad_func(X_i, y_i, theta0)
        gamma_next = theta0 - eta * g
        gamma_s_next = theta0 - s * eta * g
        theta0[:] = gamma_next + beta * (gamma_s_next - gamma_s)
        gamma[:] = gamma_next
        gamma_s[:] = gamma_s_next
```


First-Order Methods - Adaptive Learning Rate Methods

AdaGrad

AdaGrad

Update formula:

$$\text{AdaGrad} : \begin{cases} \gamma_{t+1} = \gamma_t + (g(\theta_t; \xi_t))^2 \\ \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\gamma_t + \epsilon}} g(\theta_t; \xi_t) \end{cases}$$

where

- ϵ helps control numerical stability
- Automatically rescales the gradient along each parameter direction.

First-Order Methods - Adaptive Learning Rate Methods

AdaGrad

Algorithm 4 Adagrad

Require: Initial parameters θ_0 , learning rate η

Require: Number of iterations T , stochastic gradient function $g(\theta; \xi)$

Require: small constant ϵ

- 1: $\gamma_0 \leftarrow 0, t \leftarrow 1, \theta_0 \leftarrow 0$ ▷ Initialize accumulated squared gradients
 - 2: **while** not converged **do**
 - 3: $g_t \leftarrow g(\theta_t; \xi_t)$ ▷ Compute stochastic gradient
 - 4: $\gamma_t \leftarrow \gamma_{t-1} + g_t \odot g_t$ ▷ Element-wise square accumulation
 - 5: $\theta_t \leftarrow \theta_{t-1} - \eta \frac{g_t}{\sqrt{\gamma_t + \epsilon}}$ ▷ Update parameters
 - 6: **end while**
 - 7: **return** θ_t
-

First-Order Methods - Adaptive Learning Rate Methods

AdaGrad

```
def adagrad(grad_func, theta0, eta, T, X, y, eps=1e-8):  
    gamma = np.zeros_like(theta0)  
    m = len(y)  
  
    for t in range(T):  
        i = np.random.randint(0, m)  
        X_i = X[i:i+1]  
        y_i = y[i:i+1]  
        g = m*grad_func(X_i, y_i, theta0)  
  
        gamma[:] = gamma + g**2  
        theta0[:] = theta0 - eta * g / (np.sqrt(gamma) + eps)
```

First-Order Methods - Adaptive Learning Rate Methods

AdaGrad

Problems:

- Accumulating the squared gradients over time.

First-Order Methods - Adaptive Learning Rate Methods

RMSProp

RMSProp

Update formula:

$$\text{RMSProp} : \begin{cases} \gamma_{t+1} = \alpha \gamma_t + (1 - \alpha) (g(\theta_t; \xi_t))^2 \\ \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\gamma_t + \epsilon}} \odot g(\theta_t; \xi_t) \end{cases}$$

where

- ϵ helps control numerical stability,
- α is the smooth constant
- Exponentially decay the influence of past gradients.

First-Order Methods - Adaptive Learning Rate Methods

RMSProp

Algorithm 5 RMSProp

Require: Initial parameters θ_0 , learning rate η

Require: Number of iterations T , stochastic gradient function $g(\theta; \xi)$

Require: Smoothing constant α , small constant ϵ

- 1: $\gamma_0 \leftarrow 0, t \leftarrow 1, \theta_0 \leftarrow 0$ ▷ Initialize accumulated squared gradients
 - 2: **while** not converged **do**
 - 3: $g_t \leftarrow g(\theta_t; \xi_t)$ ▷ Compute stochastic gradient
 - 4: $\gamma_{t+1} \leftarrow \alpha\gamma_t + (1 - \alpha)g_t \odot g_t$ ▷ Update running average
 - 5: $\theta_{t+1} \leftarrow \theta_t - \eta \frac{g_t}{\sqrt{\gamma_t + \epsilon}}$ ▷ Update parameters
 - 6: $t \leftarrow t + 1$
 - 7: **end while**
 - 8: **return** θ_t
-

First-Order Methods - Adaptive Learning Rate Methods

RMSProp

```
def rmsprop(grad_func, theta, eta, T, X, y, rho=0.9, eps=1e-8):  
    gamma = np.zeros_like(theta)  
    m_data = len(y)  
  
    for t in range(T):  
        i = np.random.randint(0, m_data)  
        X_i = X[i:i+1]  
        y_i = y[i:i+1]  
        g_t = grad_func(X_i, y_i, theta)  
  
        gamma[:] = rho * gamma + (1 - rho) * (g_t ** 2)  
        theta[:] = theta - eta * g_t / (np.sqrt(gamma) + eps)  
  
    return theta
```

First-Order Methods - Adaptive Learning Rate Methods

AdaDelta

AdaDelta

Update formula:

$$\text{AdaDelta : } \begin{cases} \gamma_{t+1} = \rho \gamma_t + (1 - \rho) (g(\theta_t; \xi_t))^2 \\ g'_{t+1} = \frac{\sqrt{\Delta\theta_t + \epsilon}}{\sqrt{\gamma_{t+1} + \epsilon}} \odot g(\theta_t, \xi_t) \\ \Delta\theta_{t+1} = \rho \Delta\theta_t + (1 - \rho) g_{t+1}'^2 \\ \theta_{t+1} = \theta_t - \eta g'_{t+1} \end{cases}$$

where

- ϵ helps control numerical stability
- ρ is the coefficient used for computing a running average of squared gradients

First-Order Methods - Adaptive Learning Rate Methods

AdaDelta

Algorithm 6 Adadelata

Require: Initial parameters θ_0 , learning rate η

Require: Decay rate ρ , small constant ε

- 1: $\gamma_0 \leftarrow 0, \Delta\theta_0 \leftarrow 0, t \leftarrow 0$ ▷ Initialize variables
 - 2: **while** not converged **do**
 - 3: $g_t \leftarrow g(\theta_t; \xi_t)$ ▷ Compute stochastic gradient
 - 4: $\gamma_{t+1} \leftarrow \rho \gamma_t + (1 - \rho)(g_t \odot g_t)$ ▷ Accumulate gradient energy
 - 5: $g'_{t+1} \leftarrow \frac{\sqrt{\Delta\theta_t + \varepsilon}}{\sqrt{\gamma_{t+1} + \varepsilon}} \odot g_t$ ▷ Compute scaled gradient
 - 6: $\Delta\theta_{t+1} \leftarrow \rho \Delta\theta_t + (1 - \rho)(g'_{t+1} \odot g'_{t+1})$ ▷ Update energy
 - 7: $\theta_{t+1} \leftarrow \theta_t - \eta g'_{t+1}$ ▷ Update parameters
 - 8: $t \leftarrow t + 1$ ▷ Increment iteration counter
 - 9: **end while**
 - 10: **return** θ_t
-

First-Order Methods - Adaptive Learning Rate Methods

AdaDelta

```
def adadelta(grad_func, theta, T, X, y, rho=0.95, eps=1e-6):
    gamma = np.zeros_like(theta)
    delta_theta = np.zeros_like(theta)
    m = len(y)

    for t in range(T):
        i = np.random.randint(0, m)
        X_i = X[i:i+1]
        y_i = y[i:i+1]
        g_t = grad_func(X_i, y_i, theta)

        gamma = rho * gamma + (1 - rho) * (g_t**2)
        scaled_grad = (np.sqrt(delta_theta + eps) / np.sqrt(gamma + eps)) * g_t
        theta = theta - scaled_grad
        delta_theta = rho * delta_theta + (1 - rho) * (scaled_grad**2)

    return theta
```

First-Order Methods - Adaptive Learning Rate Methods

Adam

Adam

Update formula:

$$\text{Adam} : \begin{cases} m_{t+1} = \beta_1 m_t + g(\theta_t; \xi_t) \\ \gamma_{t+1} = \beta_2 \gamma_t + (1 - \beta_2)(g(\theta_t; \xi_t))^2 \\ \theta_{t+1} = \theta_t - \eta \frac{m_{t+1}}{1 - \beta_1^t} \frac{1}{\sqrt{\frac{\gamma_t}{1 - \beta_2^t} + \epsilon}} \end{cases}$$

where

- $0 \leq \beta_1, \beta_2 < 1$: coefficients used for computing running averages of gradient and its square

First-Order Methods - Adaptive Learning Rate Methods

Adam

Algorithm 7 Adam

Require: Initial parameters θ_0 , learning rate η

Require: Exponential decay rates β_1, β_2 , small constant ϵ

- 1: $m_0 \leftarrow 0, \gamma_0 \leftarrow 0, t \leftarrow 0$ ▷ Initialize variables
 - 2: **while** not converged **do**
 - 3: $g_t \leftarrow g(\theta_t; \xi_t)$ ▷ Compute gradient
 - 4: $m_{t+1} \leftarrow \beta_1 m_t + (1 - \beta_1) g_t$ ▷ Update biased first moment
 - 5: $\gamma_{t+1} \leftarrow \beta_2 \gamma_t + (1 - \beta_2)(g_t \odot g_t)$ ▷ Update biased second moment
 - 6: $\hat{m}_{t+1} \leftarrow \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \hat{\gamma}_{t+1} \leftarrow \frac{\gamma_{t+1}}{1 - \beta_2^{t+1}}$ ▷ Bias-corrected estimates
 - 7: $\theta_{t+1} \leftarrow \theta_t - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{\gamma}_{t+1} + \epsilon}}$ ▷ Update parameters
 - 8: $t \leftarrow t + 1$ ▷ Increment iteration counter
 - 9: **end while**
 - 10: **return** θ_t
-

First-Order Methods - Adaptive Learning Rate Methods

Adam

```
def adam(grad_func, theta, eta, T, X, y, beta1=0.9, beta2=0.999, eps=1e-8):
    m = np.zeros_like(theta)
    gamma = np.zeros_like(theta)
    m_data = len(y)

    for t in range(1, T+1):
        i = np.random.randint(0, m_data)
        X_i = X[i:i+1]
        y_i = y[i:i+1]
        g_t = grad_func(X_i, y_i, theta)

        m[:] = beta1 * m + (1 - beta1) * g_t
        gamma[:] = beta2 * gamma + (1 - beta2) * (g_t ** 2)

        m_hat = m / (1 - beta1**t)
        gamma_hat = gamma / (1 - beta2**t)

        theta[:] = theta - eta * m_hat / (np.sqrt(gamma_hat) + eps)

    return theta
```

Table of Contents

- 1 Preliminaries
 - Notation
 - Mathematical Formulas
 - Loss Gradient Function
- 2 First-Order Methods
 - Gradient Descent
 - Momentum-Based Methods
 - Adaptive Learning Rate Methods
- 3 Second-Order Methods
 - Newton's Method
 - Quasi-Newton Methods
- 4 Future Work

Second-Order Methods - Newton's Method

Newton's Method

Newton's Update Formula

$$\theta_{t+1} = \theta_t - H^{-1}(\theta_t; \mathcal{D}) \nabla \mathcal{L}(\theta_t; \mathcal{D})$$

where:

- $H(\theta_t; \mathcal{D}) = \nabla^2 \mathcal{L}(\theta_t; \mathcal{D})$, the Hessian matrix of the loss function $\mathcal{L}(\theta_t; \mathcal{D})$ with respect to θ_t .

Second-Order Methods - Newton's Method

Newton's Method

Algorithm 8 Newton's Method

Require: Initial parameters θ_0 , maximum iterations T

Require: Objective function $J(\theta; \mathcal{D})$, dataset $\mathcal{D}$

```
1:  $t \leftarrow 0$ 
2: while not converged do
3:   Compute gradient:  $g_t \leftarrow \nabla J(\theta_t; \mathcal{D})$ 
4:   Compute Hessian:  $H_t \leftarrow \nabla^2 J(\theta_t; \mathcal{D})$ 
5:   Update parameters:  $\theta_{t+1} \leftarrow \theta_t - H_t^{-1} g_t$ 
6:    $t \leftarrow t + 1$ 
7: end while
8: return  $\theta_t$ 
```

Second-Order Methods - Quasi-Newton Methods

BFGS

Formula:

$$\theta_{t+1} = \theta_t - \eta_t B_t^{-1} \nabla \mathcal{L}(\theta_t; \mathcal{D})$$

- B_t is pseudo-Hessian matrix Hessian $H_t = \nabla^2 \mathcal{L}(\theta_t; \mathcal{D})$.

Second-Order Methods - Quasi-Newton Methods

BFGS

Algorithm 9 BFGS Algorithm

Require: Initial parameters θ_0 ,

Require: initial inverse Hessian approximation $B_0^{-1} = I$, learning rate η_t

- 1: $t \leftarrow 0$
 - 2: **while** not converged **do**
 - 3: Compute gradient: $g_t \leftarrow \nabla J(\theta_t; \mathcal{D})$
 - 4: Update parameters: $\theta_{t+1} \leftarrow \theta_t - \eta_t B_t^{-1} g_t$
 - 5: Compute $s_t \leftarrow \theta_{t+1} - \theta_t$
 - 6: Compute $y_t \leftarrow g_{t+1} - g_t$
 - 7: $B_{t+1}^{-1} \leftarrow \left(I - \frac{s_t y_t^\top}{y_t^\top s_t} \right) B_t^{-1} \left(I - \frac{y_t s_t^\top}{y_t^\top s_t} \right) + \frac{s_t s_t^\top}{y_t^\top s_t}$
 - 8: $t \leftarrow t + 1$
 - 9: **end while**
 - 10: **return** θ_t
-

Second-Order Methods - Quasi-Newton Methods

LBFGS

Sorry, this section is too hard for us to present. We may need Mrs Hang to teach us about this.

Table of Contents

- 1 Preliminaries
 - Notation
 - Mathematical Formulas
 - Loss Gradient Function
- 2 First-Order Methods
 - Gradient Descent
 - Momentum-Based Methods
 - Adaptive Learning Rate Methods
- 3 Second-Order Methods
 - Newton's Method
 - Quasi-Newton Methods
- 4 Future Work

Future Work

- Explore more methods.
- Convergence Analysis
- Other methods of optimization.